

Module 4 Activity 1: Reading Geometries

Purpose:

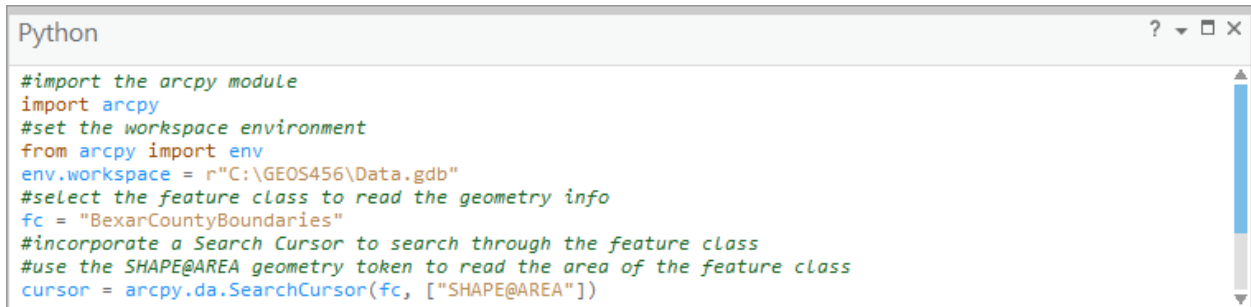
Retrieve geometry information of features in a feature class using cursors and geometry tokens

Instructions:

This exercise will be using PyScripter. Any Python environment will work.

Part 1 – Read polygon area geometry:

1. Import the **arcpy** module
2. Set the workspace
3. Select the feature class for which to read geometry information
4. Use a **SearchCursor** to search through the feature class
5. Incorporate the **SHAPE@AREA** geometry token to read the area of the feature

A screenshot of a Python script editor window titled "Python". The script contains the following code:

```
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
```

NOTE: the use of cursors are frequently used with geometries. It is important that you are familiar with the use of cursors and the parameters they accept.

6. Use the **for** loop to iterate through the records in the feature class
7. Use the **print** statement to return the area
8. Run the code and examine your result

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
#use a for loop to iterate through the feature
for row in cursor:
    Area = row[0]
    print ("Area =", Area)
```

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
#use a for loop to iterate through the feature
for row in cursor:
    Area = row[0]
    print ("Area =", Area)
Area = 0.3025287013777204
```

The area of the feature class is printed to the interpreter. However, this doesn't provide us with much useful information. We will modify the code to incorporate a **describe** function to return a spatial reference and units.

9. Add the following 2 lines of code in between the feature class and the cursor

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#use the Describe function to access the spatial reference of the feature class
spatialref = arcpy.Describe(fc).spatialReference
#specify which property you want to use
srName = spatialref.Name
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
```

The **SpatialReference** class is being used with the **Describe** function. The **Name** property is used to return the name of the spatial reference. Note that the property in relation to the spatial

reference can be impacted by whether or not the feature is projected or unprojected. In this case we cannot return the actual measurement units because the data is unprojected. If the data is projected, you can use the **linearUnitName** property to return actual unit values. Reference to the properties of the **SpatialReference** class can be found here: [SpatialReference](#)

10. Use the **print** statement to return clear results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#use the Describe function to access the spatial reference of the feature class
spatialref = arcpy.Describe(fc).spatialReference
#specify which property you want to use
srName = spatialref.Name
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
#use a for loop to iterate through the feature
for row in cursor:
    Area = row[0]
    print ("Area =", Area)
    print ("Spatial Reference Name =", srName)
```

11. Run the script and review your results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "BexarCountyBoundaries"
#use the Describe function to access the spatial reference of the feature class
spatialref = arcpy.Describe(fc).spatialReference
#specify which property you want to use
srName = spatialref.Name
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@AREA geometry token to read the area of the feature class
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@AREA"])
#use a for loop to iterate through the feature
for row in cursor:
    Area = row[0]
    print ("Area =", Area)
    print ("Spatial Reference Name =", srName)
Area = 0.3025287013777204
Spatial Reference Name = GCS_WGS_1984
```

The value of the area should be the same, but adding the **Describe** function and the **spatialReference** class with the **Name** property, we can see that the feature class is in geographic and therefore not projected.

Part 2 – Read point XY geometry:

1. Import arcpy, set the workspace and select the point feature class **Crimes2009**

NOTE: be cautious when reading geometries. Large files with many records can take a lot of time to produce results!

2. Use the **SearchCursor** to search through the point records
3. Incorporate the **SHAPE@XY** geometry token to return a list of the x and y locations of each point in the feature
4. Use a **for** loop to iterate through the records
5. Use a **print** statement with formatting to return clear results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "Crimes2009"
#incorporate a Search Cursor to search through the feature class
#use the SHAPE@XY geometry token to read the XY location of each point
cursor = arcpy.da.SearchCursor(fc, ["SHAPE@XY"])
#use a for loop to iterate through the feature
for row in cursor:
    x, y = row[0]
    print (x, y)
```

6. Run the script and examine the results

This process will take some time to generate the results as there are over 180,000 records to search through.

```
Python
-98.51238377399994 29.58974310700006
-98.55542558699995 29.517420603000062
-98.53166508399994 29.455327785000065
-98.50787570299991 29.477783109000086
-98.54980750899995 29.345041854000044
-98.49229307399992 29.433866928000043
-103.51803019599993 -6.1457575969999425
-98.49106395899992 29.428947571000037
-98.53391468599995 29.574105461000045
-98.50445220099994 29.42598913900008
-98.5236125379999 29.399637217000077
-98.41026296299992 29.527879214000052
-98.65435099299992 29.424879702000055
-98.49229307399992 29.433866928000043
-98.54609392099991 29.352856556000063
-98.44940510499993 29.513797328000066
-98.30682847500004 29.58660441000005
```

Part 3 – Read polygon XY geometry:

1. Import arcpy, set the workspace and select the polygon feature class **SchoolDistricts**
2. Use the **SearchCursor** to search through the point records
3. Incorporate the **OID@** and **SHAPE@** geometry tokens to return a list of the x and y locations of each vertex in each polygon feature

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "SchoolDistricts"
#incorporate a Search Cursor to search through the feature class
#use the OID@ geometry token to read the Object ID field
#use the SHAPE@ geometry token to read the x,y location of each vertex in the polygons
cursor = arcpy.da.SearchCursor(fc, ["OID@", "SHAPE@"])
```

The purpose of using the **OID@** geometry token is isolate and distinguish which vertices are associated with each polygon. You can use the search cursor without the **OID@** geometry token, but this will print results in one long list without breaking up the results. Doing so will make it impossible to tell which polygon vertices belong to which feature.

4. Use a **for** loop to iterate through the records
5. Use a **print** statement with formatting to return clear results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "SchoolDistricts"
#incorporate a Search Cursor to search through the feature class
#use the OID@ geometry token to read the Object ID field
#use the SHAPE@ geometry token to read the x,y location of each vertex in the polygons
cursor = arcpy.da.SearchCursor(fc, ["OID@", "SHAPE@"])
#use a for loop to iterate through the feature
for row in cursor:
    #print the results
    print ("Feature:", row[0])
```

6. Start another **for** loop and use the **getPart** method
7. Use a **print** statement to return clear results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#select the feature class to read the geometry info
fc = "SchoolDistricts"
#incorporate a Search Cursor to search through the feature class
#use the OID@ geometry token to read the Object ID field
#use the SHAPE@ geometry token to read the x,y location of each vertex in the polygons
cursor = arcpy.da.SearchCursor(fc, ["OID@", "SHAPE@"])
#use a for loop to iterate through the feature
for row in cursor:
    #print the results
    print ("Feature:", row[0])
    #use another for loop to iterate through the feature using the getPart method
    for point in row[1].getPart(0): #get part will identify if the feature consists of multiple parts
        #print the results
        print ("x:", point.X, "y:", point.Y)
```

The **getPart** method is used to return only the first part of the geometry object that has an index value of 0. This becomes especially important when reading geometries for multipart polygons. In this example, because we are only using a single part feature class (regular polygon). The **getPart** in this activity will return values of just the one polygon at a time.

More information on reading polyline and polygon geometries can be found here:

[Reading polyline or polygon geometries](#)

8. Run the code and examine the results

```
Python
x: -98.41052101799994 y: 29.476303985000072
x: -98.4109930059999 y: 29.474731028000065
x: -98.41116802599993 y: 29.474371964000056
x: -98.41299303599993 y: 29.470631019000052
x: -98.41429296399991 y: 29.468630988000086
x: -98.41439303499993 y: 29.466030983000053
Feature: 8
x: -98.42008462099994 y: 29.677163143000087
x: -98.41843163399994 y: 29.673885364000057
x: -98.41020347199992 y: 29.657565452000085
x: -98.40906794699993 y: 29.65476695700005
x: -98.40552795399992 y: 29.646044036000035
x: -98.40534095699991 y: 29.645975023000062
x: -98.40484497299991 y: 29.645826964000037
x: -98.4041649799999 y: 29.64558896500006
x: -98.40286099199994 y: 29.64527496800008
x: -98.40149296099992 y: 29.645025024000063
```

The results generated returns a list of each vertex x,y location of each polygon feature. In this case there are 13 features. Scrolling through the results will show you the x,y location of each vertex within each of the 13 polygons.